1. Project Overview

Problem Statement:
Financial institutions face a growing backlog of suspicious transactions flagged by rule-based systems. Analysts manually review these alerts by collecting data from multiple sources (e.g., online portals, mainframe systems), a process that is **time-consuming, error-prone, and inefficient**. With transaction volumes rising, the manual review process cannot scale, leading to delays in detecting genuine fraud.

Need for Automation:
The goal is to **automate data collection** and **prioritize high-risk transactions** using machine learning (ML), reducing analyst workload and accelerating decision-making.

---

2. Process Analysis

Existing Workflow:
1. An analyst receives a flagged transaction.
2. **Manual Data Collection:**
- Log into 3-4 web portals (e.g., customer profiles, transaction history).
- Query mainframe systems via terminal emulators (e.g., AS400).
- Copy-paste data into Word/Excel for review.
3. **Time Breakdown:**
- Portal navigation: **8–10 minutes/transaction**.
- Data validation: **5–7 minutes/transaction**.
- Total: **15–20 minutes/transaction**.

Inefficiencies:
- Repetitive tasks dominate 80% of the workflow.
- Analysts handle only **30–40 alerts/day**, leading to backlogs.

---

3. Solution Development

Desktop Automation Tool:
- Built with **PySimpleGUI** (user interface), **Beautiful Soup/WebDriver** (web scraping), **PyAutoGUI** (GUI automation).
- **Workflow:**
1. Auto-login to portals/systems.
2. Extract transaction details and customer history.
3. Save consolidated data to a **Word document** (for analysts) and **SQL Server** (for ML training).

Significance of 50–60K Records:
- A large dataset enables robust ML model training, capturing patterns in both

fraudulent and legitimate transactions.

---

4. Machine Learning Model

Data Preparation

- **Label Conversion:** Convert labels (e.g., "fraud"/"not fraud") to numerical IDs (0/1).
- **Train/Test Split:** 80% training, 20% validation/test.
- **Dataset Format:** Use HuggingFace `Dataset` objects for seamless integration with ML libraries.

Tokenization

- **RoBERTa Tokenizer:** Breaks text (e.g., transaction notes) into subwords.
- **Settings:**
  - `max_length=512` (truncate/pad sequences to this length).
  - `padding="max_length"`, `truncation=True` (standardize input size).

Model Architecture

- **Base Model:** Pre-trained **RoBERTa** (exceles at text understanding).
- **Classification Head:** Added on top to predict fraud probability (2 output classes).

Training Configuration

- **Batch Size:** 16 (fits GPU memory).
- **Learning Rate:** 2e-5 (avoids overwriting pre-trained knowledge).
- **Epochs:** 3–4 (prevents overfitting).
- **Mixed Precision Training:** Speeds up training with NVIDIA GPUs.

Metrics

- **Accuracy:** Overall correctness.
- **Macro F1-Score:** Balances precision/recall across imbalanced classes.

Training Process

- **HuggingFace Trainer API:** Handles gradient accumulation (simulates larger batches) and logging (track loss/metrics).

Model Saving

- Save model weights, tokenizer, and label mappings for reproducibility.

---

5. Validation Process

1. **Load Model:** Use saved weights to initialize the model.
2. **Test Data Preparation:** Tokenize test data identically to training.
3. **Custom Trainer:** Captures raw predictions (logits) for metric calculation.
4. **Prediction & Metrics:**
   - **Classification Report:** Precision, recall, F1 per class.
   - **Confusion Matrix:** Visualize false positives/negatives.

---

6. Deployment and Monitoring

Deployment Steps:
1. Package model into a Docker container.
2. Expose as an API (e.g., FastAPI/Flask) for real-time predictions.
3. Deploy on cloud (AWS/GCP) with auto-scaling.

Monitoring:
- Track **latency**, **throughput**, and **model drift** (e.g., Prometheus/Grafana).
- Log predictions to detect anomalies (e.g., sudden drop in F1-score).

---

7. Handling Data Drift and Re-Training

- **Detect Drift:** Statistical tests (e.g., Kolmogorov-Smirnov) on feature distributions.
- **Re-Training:**
  1. Triggered when drift exceeds a threshold.
  2. Fine-tune model on new data + historical data to retain prior knowledge.

---

8. Scaling the Solution

- **Horizontal Scaling:** Add more API servers behind a load balancer.
- **Batch Processing:** Use Spark/Dask for large overnight inference jobs.
- **Database Optimization:** Sharding/partitioning in SQL Server for faster queries.

Example Workflow

1. An analyst opens the desktop tool, which auto-fetches transaction data.
2. The ML model assigns a risk score (e.g., 0.95 = high risk).
3. High-risk cases are prioritized; low-risk cases are auto-approved.
4. Analysts focus on 10–15 high-risk alerts/day instead of 40 low-value ones.

Conclusion

By automating data collection and deploying an ML model, the solution reduces review time by **60–70%**, clears backlogs, and adapts to evolving fraud patterns through continuous monitoring. This end-to-end pipeline transforms a reactive process into a proactive, scalable system.